

SESSION 5: KEEPING THE WORKING ENVIRONMENT

Capturing the working environment with conda and apptainer

Kosmas Hench

2024-03-08

Finally, our reviewer is happy and convinced by the claims of our “Figure 2”. In fact, they are totally excited and want to repeat the analysis on their own data. Given all the information we have provided, this should now be easy, right? After all, they have access to the code, and they can take our workflow, pluck in their own data, press “play” to re-run the analysis and compare the results. Sure, they will need to install all the required software first, but as all the version numbers have been documented, this should be doable. Yet, as they set out to set up their system, they realize: The first step runs a script written in `python 2.3`, followed by another one written in `python 3.13`. So, which one to install? Also, after this, there is a step that requires an old custom linux program that looks like from the 90s, but the reviewer is using a new windows machine. Puzzled, they start wondering if it will be able to recreate the needed working environment after all...



created using daley

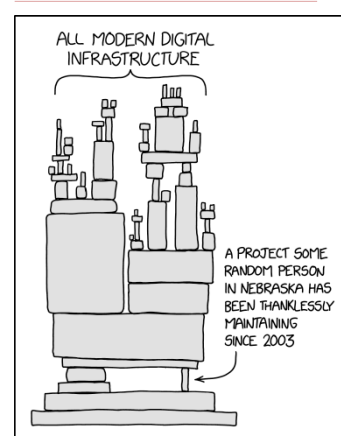
1. VERSION NUMBERS WILL NOT SAVE YOU

Just like a snowflake, your working environment is beautiful and unique. Sure, there might be many that appear similar, but deep down they are not the same — none will ever be like yours again.

A typical approach to describe the used working environment is to report of the version numbers of the programs applied throughout an analysis. Unfortunately though, the executed programs are only the tip of the iceberg: Every piece of software comes with a *dependency tree*, describing the network of additional programs and libraries it needs to be able to run. Therefore, when installing a program, one will likely install a whole stack of software. Usually, what will be installed is the most recent compatible version for each piece of this stack. This makes the exact configuration of the resulting environment dependent on the installation date, on the availability of previously installed software, as well as on the operating system (OS). Therefore, any attempt to precisely recreate the used environment at a later point in time solely based on the version numbers of the actual target programs is bound to fail.

To answer this issue, we need some means to *encapsulate the whole environment* if we want to enable others to rerun our analysis under the same conditions. We have encountered this idea before in SESSION 3, where we discussed how to record the complete used library of R packages — now, we are going to generalize this idea to an entire OS. The hope is that later it will be able to easily restore the encapsulated environment containing the matching versions of the analysis software as well as all their dependencies with a single command. Ideally, we want to avoid anyone having to go through the horrible task of working through the dependency tree and scraping together the plethora of needed programs and having to manually install them.

Besides the option of packing and shipping a working environment, encapsulating a set of installed programs unlocks another useful feature: By encapsulating a working environment, we achieve some containment, which allows us to use conflicting



Randall Munroe from [XKCD](#) illustrates the complexity of the dependency tree.

versions of the same program in different steps of an analysis. This can be very useful when the features of versions of a program differ, e.g. after an update introduces breaking changes in its behavior (Brito *et al.*, 2020). In encapsulated environments programs can be installed isolated from those located in other parallel environments. And no one is restricting you on *how many* individual working environments to use in your analysis. This makes it possible to use and combine sets of conflicting programs that would interfere with each other if they were installed system-wide.

Generally, there are two main approaches towards capturing working environments: In this workshop we are going to discuss the use of `conda` to manage virtual environments (conda community, 2024), and `apptainer` as an example of a containerization technology (Kurtzer *et al.*, 2017).

2. DESCRIBE YOUR ENVIRONMENT WITH `conda`



If you want others to be able to experience the cake you are eating, one option is to send them the precise recipe for your cake. This is what `conda` does for your working environment...

At its core, `conda` is a `python`-based package manager. Its main job is to ease the installation of software (similar to `homebrew` for mac, or `apt` in the linux world). The beauty of `conda` is that it is cross-platform (working in windows, mac and linux), and that it does not require admin privileges to install programs. These two features make it possible to easily create the same environment on quite different platforms (e.g. someone's personal macbook or a high-performance computing cluster (HPC)). Particularly for systems where users have restricted permissions, the availability of a package manager that is capable of tracking and installing programs with lots of dependencies with a single command is already a big win.

Yet, `conda` really shines when we start to use multiple `conda` environments. These are exactly the kind of encapsulated environments that we talked about before. `conda` allows you to create distinct environments and to toggle which of these is active at a given time. When using `conda` in conjunction with a workflow manager like `snakemake`, it is possible to assign individual steps of the workflow to specific `conda` environments. This combination is a great boost for reproducibility, as it explicitly binds the executed code to a defined working environment.

However, to really help others, we need to take care *how* exactly we create the `conda` environment. Similarly to the issue of data provenance, we need to keep track of what we install into a `conda` environment, and in which order. So, while it is generally possible to interactively cobble together an environment, best practice is to use and provide a recipe for it: For `conda` environments, these come in the form a `yaml` file that lists all programs (and their version numbers) that to be installed. This file can then be shipped with the code, allowing others to set up the respective environments in precisely the same way as they were build for the original analysis.

3. EXPORT AN ENTIRE OPERATING SYSTEM WITH `apptainer`

An alternative approach for encapsulating environments is the use of containers. While `conda` is a neat way to install software to any OS, containers “go the whole hog” and bake an entire OS into a single file.

This approach was pioneered with the software `docker`, which was developed for the usage within production in industry settings. This heritage still shows and makes `docker` somewhat suboptimal for usage within academic research. In this workshop we are therefore going to focus on `apptainer` (formerly known as `singularity`), which is a containerization implementation specifically tailored to the usage within research*. Generally, the usage of containers often feels a little bulkier than that of

a `conda` environment, but there are several advantages that come with the use of containers.

Plug and play. As mentioned above, a container is an OS wrapped in a single file. Whatever is installed into the container during its creation is physically contained therein. This means, that from the moment on when you download a container, you have the required software ready at your fingertips. The fact that the actual software is contained within a container makes this approach a good deal more future-proof. By comparison, `conda` requires some infrastructure to provide all needed dependencies and set up an environment. When providing a `conda` environment specification, you are banking on the hope that this infrastructure will still be in place when someone tries to re-run your code in the future.

Legacy software. Also, since containers are encompassing a whole OS, it is possible to install any software into them. Compared to `conda`, you are not restricted to what is available within a particular catalog. Basically, you can set up a container similarly like you would set up a new computer. And since you can choose which OS your container should be based upon, this is a great way to conserve legacy software that was developed for setups that are by now outdated. This is of particular value in the academic setting where much custom and unmaintained software exists. By wrapping these into a container we can greatly increase their future runnability. Obviously, this is important for the reproducibility of your code, if your analysis depends on this type of programs.

Distribution through dockerhub. Since a container is much larger than the config file describing an equivalent `conda` environment, they usually not bundled with the code. Instead, there exist dedicated platforms for the distribution of containers. Analog to github for code, there exist `dockerhub` or `quay` for sharing you own containers. These platforms are targeted at `docker` containers, which use a different file format than `apptainer` containers. Luckily, `apptainer` is capable of running `docker` containers, so this is the more widely applicable format for distribution.

Extending existing Containers. When creating a new container, you can decide what you want to base that container on. Often this may be a plain version of any operating system, but it could also be another already existing container. This means that it is easy to extend and adapt already existing containers. You can use them as a basis to spin up your own container, where you can add to the OS that is configured in your base container by installing additional software.

* The main distinction lies in the required user privileges: `docker` requires admin rights, which prohibits its usage in most HPC settings. `apptainer` can be run by any user, which makes it a little more flexible.



...another way to share the cake-eating experience is to send the actual cake. Containers like those used by `apptainer` contain an entire OS (including all installed software) within a single shareable file.

Recap

To ensure that shared code behaves the same way when someone else executes it, it is helpful to also **share the used working environment**. Two approaches to do so, are to share `conda environments` or `apptainer` containers.

→ From a reproducibility perspective, `conda environments` allow you to provide the analysis code with reference to the matching working environment, as well as a **reproducible description of the working environment**.

→ Compared to this, containers allow you to provide the analysis code alongside **the actual working environment itself**.

REFERENCES AND FURTHER READING

Brito, A. *et al.* (2020). You broke my code: Understanding the motivations for breaking changes in APIs. *Empirical Software Engineering*, 25(2):1458–1492. DOI: [10.1007/s10664-019-09756-z](https://doi.org/10.1007/s10664-019-09756-z).

conda community (2024). Conda Documentation – conda 24.9.2 documentation. [Online; accessed 25. Oct. 2024].

Kurtzer, G.M., Sochat, V. and Bauer, M.W. (2017). Singularity: Scientific containers for mobility of compute. *PLOS ONE*, 12(5):e0177459.